

Report
v. 1.0

Customer
Enso



Smart Contract Audit

TGE

13th October 2025

Report prepared by
ABDK
Consulting

Contents

1	Changelog	4
2	Summary	5
3	System overview	6
4	Methodology	8
5	Our findings	9
6	Major Issues	10
	CVF-1. FIXED	10
7	Moderate Issues	11
	CVF-2. FIXED	11
	CVF-3. FIXED	11
	CVF-4. FIXED	12
	CVF-5. FIXED	12
	CVF-6. FIXED	13
	CVF-7. FIXED	13
	CVF-8. FIXED	14
	CVF-9. FIXED	14
	CVF-10. INFO	14
8	Recommendations	15
	CVF-11. INFO	15
	CVF-12. INFO	15
	CVF-13. FIXED	15
	CVF-14. FIXED	16
	CVF-15. INFO	16
	CVF-16. INFO	16
	CVF-17. INFO	17
	CVF-18. INFO	17
	CVF-19. INFO	18
	CVF-20. FIXED	18
	CVF-21. INFO	18
	CVF-22. INFO	19
	CVF-23. INFO	19
	CVF-24. FIXED	19
	CVF-25. FIXED	19
	CVF-26. INFO	20
	CVF-27. INFO	20
	CVF-28. INFO	20
	CVF-29. INFO	20

CVF-30. INFO	20
CVF-31. INFO	21
CVF-32. INFO	21
CVF-33. FIXED	21
CVF-34. FIXED	21
CVF-35. INFO	22
CVF-36. INFO	22
CVF-37. INFO	22
CVF-38. FIXED	22
CVF-39. INFO	23
CVF-40. INFO	23
CVF-41. INFO	23
CVF-42. INFO	23
CVF-43. INFO	24
CVF-44. INFO	24
CVF-45. INFO	24

1 Changelog

#	Date	Author	Description
0.1	13.10.25	A. Zveryanskaya	Initial Draft
0.2	13.10.25	A. Zveryanskaya	Minor revision
1.0	13.10.25	A. Zveryanskaya	Release

2 Summary

All modifications to this document are prohibited. Violators will be prosecuted to the full extent of the U.S. law.

This document presents the results of a smart contract audit performed by ABDK Consulting for the System, conducted by Mikhail Vladimirov and Dmitry Khovratovich from 27 September to 10 October 2025. The goal was to assess correctness and safety of token minting and distribution, staking lifecycle and reward accounting, airdrop Merkle claims, vesting and revocation logic, access control, and upgradeability, and to identify arithmetic and procedural flaws highlighted in the issues list.

Our conclusion is that the System demonstrates good maturity and code quality. The architecture is modular, relies on OpenZeppelin upgradeable primitives (ERC20/Permit/Votes/Pausable, ERC721, UUPS, Ownable2Step). Post-remediation, we observed sound access control, precise upgrade gates, and clear staking state transitions with ERC-7201 storage segregation.

Overview of important findings: one Major issue in Airdrop assembly hashing of the to address, where high bits could taint the leaf computation, now fixed ([CVF-1](#)). Several Moderate risks from phantom overflows in proportional math in staking and minting were addressed by switching to mul/div patterns ([CVF-5](#), [CVF-9](#)). Additional Moderate procedural issues covered duplicated external calls, check ordering before reward collection, and minor naming/typing inconsistencies. The recommendations targeted gas and UX improvements (event consolidation, precomputation) and the upgradeability storage constant style.

General recommendations: continue using precise mul/div helpers for proportional calculations; minimize repeated external calls within loops; enforce early validation before state updates; keep event schemas stable and indexed fields purposeful; and maintain consistent ERC-7201 storage patterns for upgradeability. With the applied fixes and routine test coverage, we consider the System suitable for mainnet deployment, contingent on standard final checks (config parameters, role assignments, and distribution schedules) and ongoing monitoring of validator and reward flows.

It is important to note that a security audit is not a guarantee of absolute security but rather a snapshot of the system's security posture at a specific point in time. While we strive to uncover all potential issues, the evolving nature of blockchain technology and DeFi means new vulnerabilities can emerge.

3 System overview

This section provides a high-level overview of the System for token generation, distribution, staking, airdrop-based allocation, and vesting. Our team assessed how the components coordinate initial and ongoing token flows, enable participants to commit tokens for time-weighted participation with validator delegation, distribute issuance between validators and stakers, allow Merkle-based claims with optional auto-staking, and enforce beneficiary vesting with optional revocation. The System separates responsibilities and ensures that the entire token lifecycle, from initial distribution through long term incentives, is enforced on chain with clear ownership and upgrade controls.

We reviewed the [original code](#) and the [fixes](#) provided in the following files.

/		
EnsoVestingWallet.sol	EnsoVestingFactory.sol	EnsoTokenV2.sol
EnsoStaking.sol	EnsoMint.sol	Airdrop.sol
upgradeable/		
EnsoStaking Upgradeable.sol	MinterUpgradeable.sol	

At its core, the System comprises several complementary components. The staking module, **EnsoStaking** (built on an upgradeable base), models a participant's commitment as a transferable position similar to a membership certificate. Users deposit tokens into the System and select a lock period within permitted bounds; longer commitments are granted higher weighting, which increases a user's share of future rewards. Each position records who the participant delegates to, how much was deposited, the derived weight, and the time when the commitment ends. Positions may be topped up and transferred, and reward accounting is updated around ownership changes so that the rightful economic owner receives the accrued incentives. Validator administration is controlled by ownership: validators can be added and removed, operator addresses can be rotated through a pending-and-finalize flow to reduce operational risk, and validator fee adjustments are subject to a mandatory delay to prevent abrupt changes that could disadvantage participants. Reward distribution is proportional to the total delegated weight behind each validator, with participants able to accrue and later withdraw the amounts to which they are entitled.

The issuance controller, **EnsoMint**, regulates new token creation over time. It calculates the newly issued amount over a given period from a configured rate, then divides that amount among validators according to provided shares. Where a validator has no delegated stake, the validator receives the full allocation directly. Where delegation exists, the validator receives a defined fee portion and the remainder is routed into the

staking System for the benefit of the participants who delegated to that validator. This structure aligns issuance with active participation and validator service while maintaining predictable parameters for governance to adjust through ownership.

The **Airdrop** component enables distribution based on a Merkle tree. A claimant presents a proof for an indexed allocation. Upon validation, the System either transfers tokens to the recipient or, if requested by including a lock parameter and a selected validator, attempts to open a staking position on the recipient's behalf. If the position opening does not succeed for any reason, the System falls back to a direct transfer to avoid trapping funds or penalizing recipients due to integration contingencies. This mechanism allows initial allocations to be directed immediately into long-term participation when desired, improving alignment between early distribution and staking incentives.

The token module, **EnsoTokenV2**, is an upgradeable ERC-20 configured for governance and operational safety. It supports pausing to temporarily halt transfers when necessary, signature-based approvals to reduce transaction overhead for users, and vote tracking to support on-chain governance workflows. A dedicated minter role, provided by a base minter module and controlled by ownership, limits who can create new tokens. Initialization supports a one-time distribution across defined accounts through a **Distribution** list, allowing controlled initial supply setup before the token's transfer state is unpaused under owner control.

For vesting and contributor management, **EnsoVestingFactory** creates individualized **EnsoVestingWallet** contracts for beneficiaries. Each wallet enforces linear vesting with a cliff: prior to the cliff no amount is available; after the cliff, the vested amount increases continuously until the end of the schedule. Beneficiaries can release what has vested at any time. Where a revoker is configured at creation, vesting can be revoked in a way that first releases what is already earned to the beneficiary and then returns the remainder to a designated receiver. Transfers into each wallet are executed at creation so that funds are isolated and governed solely by the schedule and ownership of that specific wallet.

Across the System, access control is consistent and explicit: administrative actions such as validator management, parameter updates, token pause/unpause, minter configuration, and upgrades are reserved for the owner or designated roles. State transitions are structured to update rewards before material changes, preserving accurate accounting for participants. Upgradeability boundaries are clearly enforced, and storage is laid out to maintain safety across versions. Together, **EnsoMint**, **EnsoStaking**, **Airdrop**, **EnsoTokenV2**, **EnsoVestingFactory**, and **EnsoVestingWallet** implement a cohesive on-chain lifecycle for issuance, participation, distribution, and long-term alignment under the System.

4 Methodology

The methodology is not a strict formal procedure, but rather a selection of methods and tactics combined differently and tuned for each particular project, depending on the project structure and technologies used, as well as on client expectations from the audit.

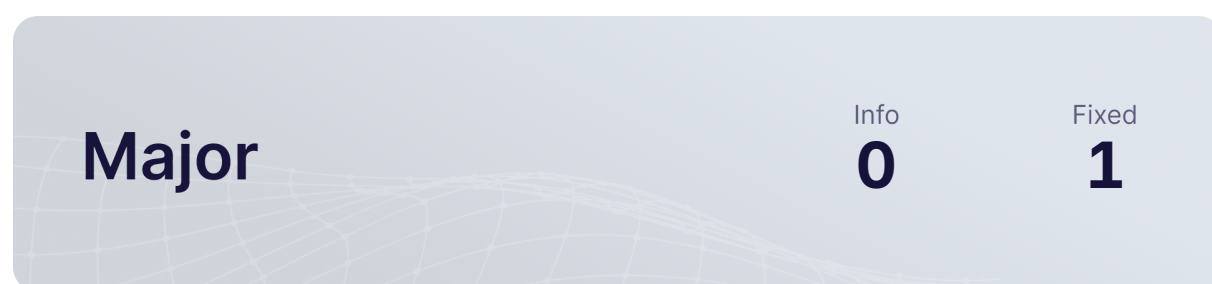
- **General Code Assessment.** The code is reviewed for clarity, consistency, style, and for whether it follows best code practices applicable to the particular programming language used. We check indentation, naming convention, commented code blocks, code duplication, confusing names, confusing, irrelevant, or missing comments etc. At this phase we also understand overall code structure.
- **Entity Usage Analysis.** Usages of various entities defined in the code are analysed. This includes both: internal usages from other parts of the code as well as potential external usages. We check that entities are defined in proper places as well as their visibility scopes and access levels are relevant. At this phase, we understand overall system architecture and how different parts of the code are related to each other.
- **Access Control Analysis.** For those entities, that could be accessed externally, access control measures are analysed. We check that access control is relevant and done properly. At this phase, we understand user roles and permissions, as well as what assets the system ought to protect.
- **Code Logic Analysis.** The code logic of particular functions is analysed for correctness and efficiency. We check whether the code actually does what it is supposed to do, whether the algorithms are optimal and correct, and whether proper data types are used. We also make sure that external libraries used in the code are up to date and relevant to the tasks they solve in the code. At this phase we also understand data structures used and the purposes they are used for.

We classify issues by the following severity levels:

- **Critical issue** directly affects the smart contract functionality and may cause a significant loss.
- **Major issue** is either a solid performance problem or a sign of misuse: a slight code modification or environment change may lead to loss of funds or data. Sometimes it is an abuse of unclear code behaviour which should be double checked.
- **Moderate issue** is not an immediate problem, but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain business flows.
- **Recommendations** contain code style, best practices and other suggestions.

5 Our findings

We found 1 major, and a few less important issues. All identified Major issues have been fixed.



Fixed 1 out of 1 issues

6 Major Issues

CVF-1 **FIXED**

- **Category** Flaw

- **Source** Airdrop.sol

Description This assumes the higher bits of "to" are set to zero, which Solidity documentation explicitly says this isn't guaranteed. See warning here: <https://docs.soliditylang.org/en/v0.8.30/assembly.html#access-to-external-variables-functions-and-libraries>

Recommendation Clear the higher bits of "to" before hashing or pack the value tightly, i.e. without padding bits.

```
49 mstore(add(ptr, 0x20), to)
```

7 Moderate Issues

CVF-2 FIXED

- **Category** Suboptimal

- **Source**

EnsoStakingUpgradeable.sol

Description The “nextPositionId” property is read from the storage twice: once here and another time inside the “nextPositionId” call.

Recommendation Refactor the code to read this value only once.

```
128 positionId = nextPositionId();  
130 EnsoStakingStorage storage $ = _getEnsoStakingStorage();  
    $.nextPositionId++;
```

CVF-3 FIXED

- **Category** Procedural

- **Source**

EnsoStakingUpgradeable.sol

Description These function are very similar.

Recommendation Consider extracting common logic into a utility function.

Client Comment *We dropped the redeem function since it is not longer functionally different after resetting the position when it has expired.*

```
173 function redeem(uint256 positionId, uint256 amount, address receiver  
    ↪ ) external {  
198 function withdraw(uint256 positionId, uint256 amount, address  
    ↪ receiver) external {
```

CVF-4 FIXED

- **Category** Procedural

- **Source**

EnsoStakingUpgradeable.sol

Recommendation These checks should be done earlier: before collecting rewards.

```
181 if (block.timestamp < position.expiry) revert PositionNotExpired(
    ↪ position.expiry, uint64(block.timestamp));
    if (amount > position.stake) revert InsufficientStake(position.stake
    ↪ , amount);

206 if (block.timestamp < position.expiry) revert PositionNotExpired(
    ↪ position.expiry, uint64(block.timestamp));
    if (amount > position.deposit) revert InsufficientDeposit(position.
    ↪ deposit, amount);
```

CVF-5 FIXED

- **Category** Overflow/Underflow

- **Source**

EnsoStakingUpgradeable.sol

Description Phantom overflow is possible here, i.e. a situation when the final calculation result would fit into the destination type, while certain intermediary calculation overflows.

Recommendation Use the "muldiv" function.

```
185 uint256 funds = position.deposit * amount / position.stake;

210 uint256 stake = position.stake * amount / position.deposit;

331 $.rewardsPerStake[validatorId] += amount * PRECISION / $.
    ↪ delegateStakes[validatorId];

499 rewards = rewardDiff * position.stake / PRECISION;

539 stake = amount + amount * $.maxMultiplier * range / ($.maxRange *
    ↪ PRECISION);
```

CVF-6 FIXED

- **Category** Bad naming

- **Source**

EnsoStakingUpgradeable.sol

Description The name of the "isSenderChecked" argument is very confusing, as one could think that "true" value means "sender is already checked, thus no need to check again" while actually the meaning is the direct opposite.

Recommendation Rename the argument to "checkSender".

```
452 function _getAccount(uint256 positionId, bool isSenderChecked)
    ↪ internal view returns (address account) {
```

CVF-7 FIXED

- **Category** Procedural

- **Source**

EnsoStakingUpgradeable.sol

Description The only place where this function is called is within the "_collectRewards" function, and according to this comment, this function shouldn't only be called after collecting rewards.

Recommendation Consider merging the logic of this function into the "_collectRewards" function.

```
502 // NOTE: must always be called after collecting rewards for this
    ↪ position or risk locking funds on this contract
function _resetPosition(address account, Position storage position)
    ↪ internal {
```



CVF-8 FIXED

- **Category** Suboptimal

- **Source** EnsoMint.sol

Description The same external contract is called many times inside the same transaction. This is inefficient.

Recommendation Refactor the code to batch multiple calls into a single call.

```
59 if (!STAKING.isValidator(validatorId)) revert NotValidator(  
    ↪ validatorId);  
60 address validator = STAKING.validatorAddress(validatorId);  
  
65 if (STAKING.delegateStake(validatorId) > 0) {  
    uint256 validatorShare = amount * STAKING.validatorFee(  
        ↪ validatorId) / _PRECISION;  
  
71 STAKING.issueRewards(validatorId, stakersShare);
```

CVF-9 FIXED

- **Category** Overflow/Underflow

- **Source** EnsoMint.sol

Description Phantom overflow is possible here, i.e. a situation when the final calculation result would fit into the destination type, while certain intermediary calculation overflows.

Recommendation Use the "muldiv" function.

```
63 uint256 amount = totalFunds * shares / totalShares;  
  
68 uint256 stakersShare = amount - validatorShare;
```

CVF-10 INFO

- **Category** Procedural

- **Source** EnsoVestingFactory.sol

Description Each call to "_create" transfers tokens from "msg.sender". Doing this multiple times inside a single transaction is inefficient.

Recommendation Refactor the code to transfer the total amount of tokens once.

Client Comment *It has to be done multiple times since each transfer is going to a different wallet. The proposed changes would have us transfer an additional time by transferring the total amount from the msg.sender to the contract address, but that is unnecessary.*

```
41 for (uint256 i = 0; i < schedules.length; i++) {  
    _create(schedules[i]);
```

8 Recommendations

CVF-11 INFO

- **Category** Procedural

- **Source**

EnsoStakingUpgradeable.sol

Description Consider specifying as "^{0.8.0}" unless there is something special regarding this particular version.

Recommendation Also relevant for: MinterUpgradeable.sol, Airdrop.sol, EnsoMint.sol, EnsoStaking.sol, EnsoTokenV2.sol, EnsoVestingWallet.sol, EnsoVestingFactory.sol.

Client Comment *These all use OpenZeppelin as dependencies, which uses version ^{0.8.20}.*

```
2 pragma solidity 0.8.20;
```

CVF-12 INFO

- **Category** Procedural

- **Source**

EnsoStakingUpgradeable.sol

Description We didn't review this file.

```
4 import { IEnsoStaking } from "../interfaces/IEnsoStaking.sol";
```

CVF-13 FIXED

- **Category** Suboptimal

- **Source**

EnsoStakingUpgradeable.sol

Recommendation This value could be rendered as "^{1e18}".

```
13 uint256 public constant PRECISION = 10 ** 18;
```

CVF-14 FIXED

- **Category** Suboptimal

- **Source**

EnsoStakingUpgradeable.sol

Recommendation This value could be rendered as "0.2e18".

```
14 uint96 public constant DEFAULT_FEE = 2 * 10 ** 17; // 20% fee
```

CVF-15 INFO

- **Category** Suboptimal

- **Source**

EnsoStakingUpgradeable.sol

Recommendation It would be more efficient to merge these mappings into a single mapping whose keys are validator IDs and values are structs encapsulating the values of the original mappings.

Client Comment *The proposed changes substantially increase gas costs*

```
25 mapping(bytes32 => ValidatorData) validators;
   mapping(bytes32 => address) pendingValidators;
   mapping(bytes32 => PendingValidatorFee) pendingFees;

31 mapping(bytes32 => uint256) delegateStakes; // delegate => stake
   mapping(bytes32 => uint256) rewardsPerStake; // delegate => value
   ↪ per stake
   mapping(bytes32 => uint256) totalRewards; // delegate => rewards
   ↪ earned
```

CVF-16 INFO

- **Category** Suboptimal

- **Source**

EnsoStakingUpgradeable.sol

Recommendation It would be more efficient to merge these two mappings into a single mapping whose keys are depositor addresses and values are structs encapsulating the values of the original mappings.

Client Comment *The proposed changes substantially increase gas costs*

```
29 mapping(address => uint256) depositorStakes; // depositor => stake
30 mapping(address => uint256) collectedRewards; // depositor =>
   ↪ rewards collected
```


CVF-17 INFO

- **Category** Procedural

- **Source**

EnsoStakingUpgradeable.sol

Description Solidity compiler is smart enough to precompute constant hash expressions.

Recommendation Use hash expression instead of a hardcoded hash value.

Client Comment *EnsoStakingStorageLocation gets used as a storage slot in assembly. The compiler throws the following error: 'Only direct number constants and references to such constants are supported by inline assembly'.*

```
37 // keccak256(abi.encode(uint256(keccak256("enso.storage.Staking")) -  
    ↪ 1)) & ~bytes32(uint256(0xff))  
bytes32 private constant EnsoStakingStorageLocation =  
0xab7f204927625a68eb5fa7fef90efd520ba1844238d9aca28f6f0f9cde79b00;
```

CVF-18 INFO

- **Category** Procedural

- **Source**

EnsoStakingUpgradeable.sol

Recommendation The type for this argument should be "IERC20".

Client Comment *Disagree.*

```
52 address _token,
```

```
67 address _token,
```

CVF-19 INFO

- **Category** Suboptimal

- **Source**

EnsoStakingUpgradeable.sol

Description Emitting several events for a single operation is inefficient.

Recommendation Consider merging into a single event.

Client Comment *Events like ValidatorAddressUpdated, ValidatorFeeUpdated, and FundsDeposited are called by other functions. We want a user tracking a single event to have a complete history of those event parameters.*

```
96 emit ValidatorAdded(validatorId);
emit ValidatorAddressUpdated(validatorId, validator, address(0));
emit ValidatorFeeUpdated(validatorId, DEFAULT_FEE, 0);

138 emit PositionCreated(positionId, expiry, validatorId);
emit FundsDeposited(positionId, amount, stake);
```

CVF-20 FIXED

- **Category** Suboptimal

- **Source**

EnsoStakingUpgradeable.sol

Description The value "validatorAddress(validatorId)" is calculated twice.

Recommendation Calculate once and reuse.

```
274 if (msg.sender != validatorAddress(validatorId)) {
    revert InvalidSender(validatorAddress(validatorId), msg.sender);
}
```

CVF-21 INFO

- **Category** Procedural

- **Source** MinterUpgradeable.sol

Description Solidity compiler is smart enough to precompute constant hash expressions.

Recommendation Use hash expression instead of a hardcoded hash value.

Client Comment *MINTER_STORAGE_LOCATION gets used as a storage slot in assembly. The compiler throws the following error: 'Only direct number constants and references to such constants are supported by inline assembly'.*

```
13 /// @dev keccak256(abi.encode(uint256(keccak256("enso.storage.Minter
    ↪ )) - 1)) & ~bytes32(uint256(0xff))
bytes32 private constant MINTER_STORAGE_LOCATION =
0x18f84b571f526b7af469fa4b3a4958cd3902fd46e02c00db342c2381bbdbdf00;
```



CVF-22 INFO

- **Category** Bad naming
- **Source** MinterUpgradeable.sol

Recommendation Events are usually named via nouns, such as "Minter".

Client Comment *Disagree. Events should be verbs. This name is fine.*

```
26 event MinterUpdated(address indexed previousMinter, address indexed  
    ↪ minter);
```

CVF-23 INFO

- **Category** Suboptimal
- **Source** MinterUpgradeable.sol

Description The "previousMinter" parameters is redundant, as its value could be derived from the previous event.

Recommendation Remove the "previousMiner" parameter.

Client Comment *This is common practice. For example OpenZeppelin's Ownable contract.*

```
26 event MinterUpdated(address indexed previousMinter, address indexed  
    ↪ minter);
```

CVF-24 FIXED

- **Category** Suboptimal
- **Source** MinterUpgradeable.sol

Description This check seems redundant, as it is anyway possible to pass a dead minter address.

Recommendation Remove this check.

```
34 if (newMinter == address(0)) {  
    revert MinterInvalidOwner(address(0));
```

CVF-25 FIXED

- **Category** Unclear behavior
- **Source** MinterUpgradeable.sol

Description This event is emitted even if nothing actually changed.

```
59 emit MinterUpdated(oldMinter, _newMinter);
```

CVF-26 INFO

- **Category** Procedural

- **Source** Airdrop.sol

Description We didn't review this file.

```
4 import { IEnsoStaking } from "../interfaces/IEnsoStaking.sol";
```

CVF-27 INFO

- **Category** Procedural

- **Source** Airdrop.sol

Recommendation The "to" parameter should be indexed.

```
12 event Claim(uint256 indexed index, address to, uint256 amount);
```

CVF-28 INFO

- **Category** Bad naming

- **Source** Airdrop.sol

Recommendation Events are usually named via nouns.

Client Comment *Disagree.*

```
13 event ClaimStaked(uint256 indexed index, uint256 positionId);  
event StakeFailed(uint256 indexed index, bytes error);
```

CVF-29 INFO

- **Category** Procedural

- **Source** Airdrop.sol

Recommendation The "positionId" parameter should be indexed.

```
13 event ClaimStaked(uint256 indexed index, uint256 positionId);
```

CVF-30 INFO

- **Category** Suboptimal

- **Source** Airdrop.sol

Recommendation These errors could be made more useful by adding certain parameters into them.

Client Comment *Disagree.*

```
16 error InvalidMerkleProof();
```

```
18 error AirdropExpired();  
error AirdropNotExpired();
```



CVF-31 INFO

- **Category** Bad datatype
- **Source** Airdrop.sol

Recommendation The type for the "_token" argument should be "IERC20".

Client Comment *Disagree.*

```
28 constructor(address _token, address _staking, bytes32 _root, uint256
    ↪ _expiration, address _owner) Ownable(_owner) {
```

CVF-32 INFO

- **Category** Bad datatype
- **Source** Airdrop.sol

Recommendation The type for the "_staking" argument should be "IEnsoStaking".

Client Comment *Disagree.*

```
28 constructor(address _token, address _staking, bytes32 _root, uint256
    ↪ _expiration, address _owner) Ownable(_owner) {
```

CVF-33 FIXED

- **Category** Suboptimal
- **Source** Airdrop.sol

Recommendation Right shift and bitwise AND would be more efficient here.

```
58 uint256 claimedWordIndex = index / 256;
    uint256 claimedBitIndex = index % 256;
```

```
103 uint256 claimedWordIndex = index / 256;
    uint256 claimedBitIndex = index % 256;
```

CVF-34 FIXED

- **Category** Procedural
- **Source** Airdrop.sol

Recommendation Brackets are redundant.

```
62 uint256 mask = (1 << claimedBitIndex);
```

CVF-35 INFO

- **Category** Procedural
- **Source** EnsoMint.sol

Description We didn't review these files.

```
4 import { IEnsoStaking } from "../interfaces/IEnsoStaking.sol";  
import { IEnsoToken } from "../interfaces/IEnsoToken.sol";
```

CVF-36 INFO

- **Category** Suboptimal
- **Source** EnsoMint.sol

Recommendation This value could be rendered as "1e18".

```
12 uint256 private constant _PRECISION = 10 ** 18;
```

CVF-37 INFO

- **Category** Bad naming
- **Source** EnsoMint.sol

Recommendation Events are usually named via nouns.

Client Comment *Disagree.*

```
25 event FundsIssued(uint256 amount, uint256 period);  
event TokensPerSecondUpdated(uint256 amount);
```

CVF-38 FIXED

- **Category** Suboptimal
- **Source** EnsoMint.sol

Recommendation This error could be made more useful by adding certain parameters into it.

```
30 error InvalidTimestamp();
```

CVF-39 INFO

- **Category** Bad datatype
- **Source** EnsoMint.sol

Recommendation The type for the "_token" argument should be "IERC20".

Client Comment *Disagree.*

```
32 constructor(address _token, address _staking, address _owner,  
    ↪ uint256 _tokensPerSecond) Ownable(_owner) {
```

CVF-40 INFO

- **Category** Bad datatype
- **Source** EnsoMint.sol

Recommendation The type for the "_staking" argument should be "IEnsoStaking".

Client Comment *Disagree.*

```
32 constructor(address _token, address _staking, address _owner,  
    ↪ uint256 _tokensPerSecond) Ownable(_owner) {
```

CVF-41 INFO

- **Category** Bad datatype
- **Source** EnsoStaking.sol

Recommendation The type for this field should be "IERC20".

Client Comment *Disagree.*

```
12 address _token,
```

CVF-42 INFO

- **Category** Procedural
- **Source** EnsoTokenV2.sol

Description We didn't review this file.

```
4 import { Distribution } from "../libraries/Distribution.sol";
```

CVF-43 INFO

- **Category** Bad naming
- **Source** EnsoVestingWallet.sol

Recommendation Events are usually named via nouns, such as "TokenRelease" or "VestingRevocation".

Client Comment *Disagree.*

```
13 event TokenReleased(uint256 releasedAmount);  
event VestingRevoked(uint256 forfeitedAmount, address receiver);
```

CVF-44 INFO

- **Category** Suboptimal
- **Source** EnsoVestingWallet.sol

Description This check is redundant, as it is superseded by the "msg.sender" check below.

Recommendation Remove this check.

Client Comment *No it isn't. Some vesting wallets are not revocable, regardless of who the msg.sender is.*

```
90 if (!isRevocable()) revert NotRevocable();
```

CVF-45 INFO

- **Category** Bad datatype
- **Source** EnsoVestingFactory.sol

Recommendation The type for the "_token" argument should be "IERC20".

Client Comment *Disagree.*

```
32 constructor(address _token, address _owner) Ownable(_owner) {
```




ABDK

Consulting

About us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and co-authored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 40 audits of blockchain projects in Solidity, Rust, Circom, C++, JavaScript, and other languages.

Contact

Email

dmitry@abdkconsulting.com

Website

abdk.consulting

Twitter

twitter.com/ABDKconsulting

LinkedIn

linkedin.com/company/abdk-consulting